

Lista: Predicados Numéricos.

Nome: _____ **Matrícula:** _____

Exercícios baseados no capítulo 22 do livro Modern Programming Languages.

É possível que você precise do predicado findall para resolver alguns dos exercícios abaixo. Estude um pouco sobre este predicado e veja como ele funciona.

Exercício 01. Defina um predicado `max(X, Y, Z)` que recebe os números `X` e `Y` e unifica `Z` com o maior dos dois.

Exercício 02. Defina um predicado `max_list(L, M)` que recebe uma lista `L` de números e unifica `M` com o maior número na lista. O predicado deve falhar se a lista `L` estiver vazia.

Exercício 03. Defina um predicado `ordered(L)` que verifica se uma lista de números `L` está em ordem crescente— cada elemento é menor ou igual ao próximo.

Exercício 04. Defina um predicado `merge_sort(In, Out)` que gera uma versão ordenada de uma lista `In` de números e unifica o resultado com `Out`. Seu predicado deve usar um algoritmo de `merge sort`. O SWI-Prolog possui os predicados predefinidos `sort` e `insert`, que implementam variações do `merge sort`. Não use esses predicados, mas escolha outro nome para o seu.

Exercício 05. Defina um predicado `n_queens(N, X)` que resolve o problema das `N`-rainhas. O problema das `N`-rainhas é semelhante ao problema das oito rainhas, mas requer a colocação de `N` rainhas em um tabuleiro de xadrez `NxN`.

Exercício 06. Defina um predicado `subset_sum(L, Sum, SubL)` que recebe uma lista `L` de números e um número `Sum` e unifica `SubL` com uma subsequência de `L` tal que a soma dos números em `SubL` é igual a `Sum`. Por exemplo:

```
?- subset_sum([1, 2, 2, 5, 3], 5, Subset).  
Subset = [1, 2, 2] ;  
Subset = [2, 3] ;  
Subset = [5] ;  
Subset = [3, 2] ;  
false.
```

Seu predicado deve assumir que `L` e `Sum` estão instanciadas e deve ter sucesso uma vez para cada subsequência de `L` que soma `Sum`.